

1. Uso de Frameworks y ORM

• **¿Consideras el uso de un framework una ventaja o un inconveniente? Explica los motivos. ¿Qué ORMs conoces y en que crees que ayudan o dificultan en la programación de aplicaciones? Explica brevemente los motivos que justifican el hecho de usar o no un ORM.**

Des del punt de vista d'organització d'equips l'ús de frameworks és clarament un avantatge, però amb matisos importants, des del meu punt de vista depen de cada cas d'ús

En el cas de laravel/symfony són perfectes per escalar i fer projectes reals, permeten patrons clars i l'onboarding de programadors no pot ser més directe ja que coneixen l'estructura a més incorporen eines de seguretat de base que són molt útils. L'inconvenient és el deute tècnic i l'exces de pes que tenen, has d'estar pendent de vulnerabilitats i actualitzar paquets constantment, a més per evitar que una simple consulta carregui centenars de fitxers que no utilitza, s'ha d'optimitzar amb opcache per evitar un mal rendiment, en situacions normals com un dashboard amb molta informació pot alentir el sistema de forma descontrolada si les consultes no estan ben fetes.

Per a micro-frameworks com slim que són molt pragmàtics quan només necessites l'essencial com rutes i seguretat, i recuperar el control del rendiment per fer un microservei o API específica, el motiu per el qual no els gaïu servir per a tot és que perds velocitat de desenvolupament, ja que has de configurar a més moltes eines que un framework ja dona fetes.

Finalment també he fet servir fitxers de php sense cap framework i són útils per casos molt específics com executar comandes per terminal o fer servir els recursos del servidor al màxim, em passava amb RSindústria on carregava bases de dades de molt grans fen servir directament mysqldump o executant crawlers de Python des de php.

Fer una web completa amb php pur avui és un error no te sentit ja que ràpidament es converteix en codi marginat, amb greus problemes de seguretat i fa impossible la escalabilitat si entren nous companys al equip.

En conclusió faig servir frameworks per que em permeten ser molt més àgil, treballar en equip i tenir un codi més net i sobretot no reinventar infraestructura o eines.

Sobre els ORMs

He treballat a fons amb Eloquent (laravel) i Doctrine (symfony). En el 90% dels casos són la millor opció per al dia a dia i els CRUDs específicament, ajuden a no repetir codi i mantindre el projecte molt net.

El que no m'agrada és que amaguin el codi real. Si no saps que estan fent per sota, trobar l'origen d'un problema de rendiment és molt complex. És molt fàcil tenir problemes de consultes N+1, no estan pensats per gestionar milers de registres de cop. Si intentes carregar una pantalla amb moltes estadístiques per exemple 30.000 registres l'ORM generarà 30.000 objectes php a la memòria i pot fer caure el servidor, te solució optimitzant la configuració de php però requereix atenció extra.

Com a conclusió intento mantenir tota la lògica estàndard amb l'ORM per productivitat i que el meu codi sigui net, però hi ha limitacions tècniques que per defecte s'han de controlar com eager loading o paginacions per evitar problemes de rendiment i quan no hi ha més remei en casos crítics com consultes massives en que descarto l'ús del ORM i ataco directament amb SQL pur, per que les eines que se suposa que han d'ajudar, acaben entorpint.

2. Integración con APIs Externas:

- **¿Cómo plantearías una nueva integración con una API externa? ¿Qué tipo de arquitectura, autenticación y otros elementos de seguridad tendrías en cuenta?**

Quan plantejo una integracio de una API externa(ja sigui per consumir dades de un tercer o per conectar diferents eines entre si com podria ser zoho o un erp extern) el primer que assumeixo es que passa si em falla, que pot anar lenta la conexio o tallar-se o envie dades incorrectes, per tant el principal objectiu es protegir el sistema per evitar que la caiguda de un proveedor extern no arrossegui la plataforma sencera,per això ho agrupo en els següents pilars:

La arquitectura i resiliencia:

La prioritat es no bloquejar la resposta a l'usuari, Si una accio ha de fer una crida a un process que requereix una API externa , evito fer-ho de forma sincrona un request HTTP. El que faig es registrar el event, ficar-lo en segon pla i un worker s'encarrega de gestionar-ho, aixi podem tenir molt mes control del process, per experiencia propia a vegades hi ha microtalls o simplement hi ha caigudes temporals ,si hi ha un tall, el worker esta programat per fer intents de 1s, 2s, 4s, 8s... Aixi evito bombardejar la api externa , si esta tenint problemes, amb intents instantanis el unic que fem es empetjorar-ho , que ens fiquin en blacklist per request 500 o rate-limiting.

A mes configurar timeouts estrictes i monitoritzar les peticions, per exemple si la api externa dona 10 errors 500 seguits, simplement faria una pausa de 30s abans de permetre fer crides noves. Es preferible parar la integracio temporalment si ha caigut, que esgotar recursos del nostre servidor obrint peticions que es queden penjades i no s'arriben a tancar mai esperant una resposta que no arriba.

Seguretat i autenticacio:

Per el que fa a la seguretat, que el transit vagi per HTTPS o TLS es innegociable.

A nivell d'autenticacio, si el proveedor ho permet la meua preferencia es utilitzar protocols amb tokens de curta duracio i rotacio automatica mil vegades que no pas un api key estatic.

Si no hi ha mes remei que no sigui una credencial estatica mai la ficaria hardcodejada al codi, La meua forma de treballar a nivell local es utilitzant variables d'entorn desde fitxers .env (sempre exclos al .gitignore), per a produccio prefereixo que vinguin desde un gestor de confiança com el secrets manager de AWS per evitar fitxers de text pla que puguin comprometre el servidor,

Dades i zero confiança amb el proveedor:

Finalment evito confiar en el proveedor sota ninguna circumstancia, sempre pot fallar de moltes maneres creatives i al tractarse d'un servei extern s'ha de sanejar les dades i validar amb filtres de esquema json per evitar corrupcio de dades o injeccions de codi, això ho he après a través a base d'errors per exemple passa molt que fas una crida a una api externa, es processa, pero hi ha un micro-tall a la xarxa i no reps el 200 ok, El worker ho intenta automàticament per error i duplica la crida a conseqüència, Si això passa al CRM molt probablement es duplicaria una matricula o crearia un alumne dues vegades, per evitar el duplicat i portar un control si el proveedor ho accepta l'hi assigno un ID unic amb UUID aixi si el proveedor rep el mateix ID detecta el duplicat i no repetix la accio, i quan no hi ha forma que el proveedor accepti un ID simplement ho gestiono amb el worker i la persistencia a la BBDD que he comentat abans en l'arquitectura, això ho vaig implementar en un projecte per enviar correus massius per fer que el sistema sigui robust.

i per suposat tot ha de estar monitoritzat de manera que pugui rebre alertes per taxes de errors si un endpoint puja un cert percentatge de errors per enterar'men abans que el client.

3. Creación de API

- **¿De qué forma empezarías el desarrollo de una nueva API? ¿De qué forma trabajarías la implementación de la misma considerando que hay parte de frontend y de backend?**

Per poder plantejar correctament un desenvolupament de una API el que necessitaria inicialment abans de picar res de codi seria parlar amb els equips implicats en el cas de que hi hagi front i back també necessitaria saber quin es el objectiu final per poder saber quina seria la funcionalitat i com es consumiria o quin us tindria, quan estigues tot clarament definit ho documentaria per poder començar, Com que hi ha una part de frontend(o sistemes externs com el CRM) i backend(o microserveis), l'objectiu principal seria evitar que un equip bloqueji al altre i ho faria de la següent manera:

Primer que res, abans de tocar arquitectura, planejaria la definició del contracte de dades que necessita l'equip de frontend: que necessiten ,quin volum de dades s'espera, quina recurrencia hi haurà, qui ho veura, quin tipus de dades son etc. Això em dona una mica de visió a curt i llarg termini i puc començar a definir una estructura. Un cop tinc clars quins seran els endpoints nous, crearia la documentació tècnica necessària amb Postman, definiria un contracte amb dades de prova(un json mockup), així el frontend pot començar a desenvolupar i integrar pantalles desde el minut 1 i sense esperar que el backend estigui acabat. Treballem en paral·lel, ells mostren les dades per pantalla i jo faig els tests que necessito al meu ritme.

Amb el backend abans de crear res de zero , revisaria la infraestructura i codi actual per poder prioritzar la coherència del codi i no introduir complexitat innecessària o reescriure codi. Si ja hi ha patrons que funcionen o s'adapten al que necessitem com a micro-serveis o un framework intern, els reutilitzaria com a prioritat.

A nivell de dades em centraria en validar bé tot el que entra abans que toqui la base de dades i formatjar bé el que surt per que el frontend només rebi el que ha demanat.

Per poder tenir una estructura que sigui estàndard i aguantar en un futur hi ha moltes coses que s'han de implementar però que dependrien de la orientació del projecte a continuació menciono les que he fet servir:

Crear una configuració mínima perquè l'aplicatiu capturi qualsevol excepció i retorni sempre una estructura estàndard amb els codis http correctes, així no enviaria mai HTML trencat al front si hi ha un error 500, 422 si falla la validació , 404 ... la idea és crear una api estandaritzada o fer servir algun protocol com openapi.

Tenir una estructura de versions. Sempre hi han canvis i per evitar rescriure tot el codi quan els canvis són grans, és més fàcil començar amb una estructura clara com /v1, així en un futur sempre es poden mantenir l'API legacy i fer una altra branca a la /v2 sense tancar l'aplicatiu.

Fer servir cache com redis o algun proxy intermig com un cdn, depenent del llenguatge de programació o el entorn si és un framework o microservei.

Ficar seguretat bàsica de us , límits per evitar abusos , per temps, per usuari autènticat, per ip , per sessió amb JW autoritzat, tokens amb rotació o qualsevol cosa que sigui necessària per restringir el access.

I finalment, el desplegament, encara que en els projectes de laravel/symfony/wordpress que he treballat no requeria automatitzacions de CI/CD si que he fet servir tests unitaris i d'integració per poder verificar noves implementacions i garantir que no hi hagues cap conflicte amb noves integracions, soc estricte en fer servir git per assegurar-me l'estabilitat de desenvolupament. La meua norma general és no modificar res de producció que no estigui testejat en sandbox o local amb docker per tenir el mateix sistema que a producció, i sempre mantindre snapshots tant de codi o base de dades, així es pot fer un rollback o migracions ràpides i netes. Un cop tot el desenvolupament estigues finalitzat , per entrar a producció intentaria buscar la forma de poder fer un staging o canary per poder tenir feedback a producció, abans de fer un canvi total i un cop assegurat el funcionament, simplement permetria el access total i monitoritzaria que tot estigues operatiu.

4. Arquitectura y patrones:

- **¿Qué tipo de arquitecturas conoces? ¿Para qué usarías cada una de las arquitecturas? En cuanto a patrones, ¿puedes enumerar patrones de diseño que conozcas y que hayas trabajado y casos de uso en los que se deben usar?**

Per respondre directament a la pregunta, la meua visió sobre la arquitectura i els patrons no és estricta en el sentit de seguir específicament tots i cada un dels punts teòrics sinó que en la meua experiència personal m'he trobat casos en els que he tingut que evitar seguir-ho per poder fer el projecte viable, l'objectiu és determinar quan fer un MVP o una prova concepte sense perdre 3 dies mirant tots els detalls de la arquitectura per poder demostrar la idea de negoci, i quan toca montar una estructura sòlida per poder escalar. Per poder explicar quins patrons i arquitectures conec i utilitzo m'agradaria posar com exemple el ERP que vaig desenvolupar damunt de un open source amb symfony, projectes que he fet amb laravel i altres projectes que m'han fet aprendre molt:

Sobre arquitectures:

Treballant amb laravel, parteixo del clàssic monolític basat en MVC. Per a un ERP o CRM, és més ràpid i fàcil de mantenir en la majoria de casos sempre que estigui ben separat (separant controladors de la lògica de negoci).

Utilitzo els microserveis només quan hi ha casos en els que el framework es queda curt, per exemple amb un dels projectes que he fet que consistia en fer scraping diari de 100.000 productes per e-commerce. Si posava aquest crawler dins el mateix servidor, el consum de memòria em feia caure la plataforma sencera, vaig prendre la decisió de fer un pipeline amb python separat i ho vaig convertir en un microservei independent que garanteix el funcionament amb normalitat de la resta del sistema.

Entenc la importància de l'arquitectura event-driven que crec que és fonamental per a un ecosistema com el vostre per respondre a webhooks o automatitzacions del CRM.

Sobre patrons i el codi:

Com deia abans, la base és MVC model vista controlador però es queda curt ràpidament, els "patrons" o al·lorama en la que escric codi que mencionare com exemple a continuació són els que m'han ajudat a no tenir fitxers de 1000 línies descontrolats seguint una estructura escalable i fàcil de mantenir:

Events i cues o patro observer: A comprasonline vaig dissenyar sistemes per enviar de forma massiva 15.000 correus diàriament. No podia ser un procés seqüencial, si falla un correu no es pot cancel·lar tot el enviament, per tant vaig crear un sistema per generar campanyes on es registren les dades amb el seu estat persistent a la BBDD. Un cron revisa si es compleixen les condicions llança processos de workers en segon pla totalment desacoplat. Aquests processos tenen controls d'enviament, reintents, recuperació, actualització a temps real de dades. El codi està tot separat i estructurat per cada capa fent que afegir una nova funcionalitat no canvia res del projecte principal que ja existeixen.

Aïllar crides externes o patro adapter: Quan he integrat proveïdor externs de dades grans com TecDoc via API, mai poso les crides directament al controlador. Ho vaig fer amb una simple classe a part (una mena d'Adaptador) que només serveix per comunicar-se amb TecDoc, evitant així tenir que crear un codi espagueti o un controlador gegant. D'aquesta manera si l'API canvia o s'actualitza algun camp només he de tocar un sol fitxer de tot el projecte.

Comandes CLI o patro command: A Rsindustria havia de carregar bases de dades de 40gb amb taules que tenen més de 250M de registres i fer còpies. Intentar fer això utilitzant les eines estàndards de Symfony o els ORM era completament impossible per rendiment. La meua aproximació va ser dissenyar scripts de comandes per terminal fets a mida que s'executaven fora de l'estructura del framework, optimitzat per aconseguir el màxim d'IOPS a MySQL. Amb això em refereixo que cada eina té la seva feina i a vegades també és saber quins són els límits de l'arquitectura per poder prendre millors decisions.

En resum, conec i aplico la teoria però sempre seguint l'objectiu de que el projecte sigui viable tècnica i econòmicament, documentant i treballant de forma neta per tal que qualsevol companya ho pugui modificar de forma ràpida i estructurat i que no trenqui la resta del aplicatiu amb testos i facilitant el manteniment.

5. Prueba técnica:

Realizar los endpoints de API de un CRUD de un modelo estudiante.

El modelo Estudiante, debe tener los siguientes atributos obligatorios:

- Nombre (longitud mayor de 4 caracteres y menor de 60)
- Email
- Teléfono

Además, debe contener los siguiente atributos NO obligatorios:

- Dirección
- Número de documento de identidad

Se puede realizar en el framework Laravel o Symfony.

El código se puede subir a github (o otra plataforma) o enviarlo directamente por correo con las instrucciones/documentación para su ejecución.